

HWSW Project: nbody Benchmark Report

1. Overview

Benchmark: nbody

Description: The benchmark simulates the orbital motion of the outer solar system (Sun, Jupiter, Saturn, Uranus, Neptune - 5 bodies total) using a simple integrator. Each of the 20,000 simulation steps computes the pairwise gravitational force between every pair of bodies (10 pairs for 5 bodies), updates each body's velocity accordingly, and then updates each body's position from its velocity. Before and after the simulation, the total energy of the system (kinetic + potential) is computed and used as a correctness check.

Libraries used. Original benchmark: pure Python standard library only, no external dependencies; timing/statistics handled by pyperf. Optimized (numba) version: numpy (fixed-type array storage) and numba (JIT compilation to native code).

Data structures. Original: each body is a tuple (position, velocity, mass), where position and velocity are 3-element Python lists. All 5 bodies are stored in a Python list. All 10 unique body pairs are pre-computed once at import time via a hand-written combinations() helper.

Optimized Numba version: positions, velocities and masses are stored as NumPy float64 arrays of shape (5,3), (5,3) and (5,), which allows Numba to compile the numerical loops efficiently.

2. Initial Analysis

Profiling setup: `perf record -F 999 -g -- python3-dbg -m pyperformance run --bench nbody`, followed by `perf report --stdio`, using a debug build of Python (python3-dbg) so internal CPython frames are resolved with symbols.

Findings: Profiling showed that the benchmark spends most of its time executing the main numerical loops through the CPython interpreter. The `perf` results and the baseline flame graph both show heavy interpreter activity around `advance()` and `report_energy()`.

From the source code, we initially identified two possible causes:

- the Python loop over the 10 body pairs;
- the distance calculation using `** (-1.5)`.

We tested both ideas separately. Neither improved performance, which indicated that the main bottleneck was the Python interpreter itself. This motivated the use of Numba JIT compilation.

3. Optimizations

All optimizations were measured with pyperf. Correctness of every optimized version was independently verified by comparing `report_energy()` before/after 20,000 iterations against the original's `pow()`-based, list-of-tuples implementation.

Optimization 1: Manual loop unrolling

Idea: We suspected that looping over the list of 10 body pairs and repeatedly unpacking Python objects added significant overhead.

Change: We replaced the generic pair loop with explicit code for all 10 body pairs.

Result: Runtime remained about **231 ms**, almost identical to the baseline of **229–231 ms**. `pyperf` reported that the difference was not statistically significant.

Conclusion: Python loop and tuple-unpacking overhead were not the main bottleneck.

Optimization 2: `sqrt()` instead of `** (-1.5)`

Idea: We suspected that computing distance with `** (-1.5)` was expensive.

Change: We replaced it with an equivalent calculation using `sqrt()`, division, and multiplication.

Result: Runtime increased to **239–242 ms**, around **4–5% slower** than the baseline.

Conclusion: The new version required more Python-level operations, so it became slower. This showed that changing the mathematical expression alone was not enough; Python interpreter overhead was still dominant.

Optimization 3: Numba JIT compilation (nopython mode)

Idea: Since the first two optimizations did not help, we targeted the Python interpreter overhead directly.

Change: Positions, velocities, and masses were moved to NumPy `float64` arrays, and the main computation functions were compiled using `@numba.njit`. A warm-up call was performed before timing so JIT compilation time was not included.

Result:

Baseline: ~229 ms

Numba: 9.24 ms

This is approximately a **24.8–25× speedup**.

Conclusion: The result confirms that Python interpreter overhead was the main performance limitation. Compiling the hot computation to native code produced by far the largest improvement.

4. Performance Comparison

4.1 pyperf wall-clock comparison (process-isolated measurements)

Optimization	Result	Verdict
Baseline (original)	229-231 ms +/- 3-5 ms	-
1. Unrolled	231 ms +/- 4-5 ms	Not significant
2. sqrt	239-242 ms +/- 3-8 ms	1.04-1.05x SLOWER
3. Numba	9.24 ms +/- 0.12 ms	24.82x FASTER

4.2 Flamegraph Insights

Figure 1. Flame graph of the original nbody benchmark. Most execution remains inside Python/CPython interpreter frames.

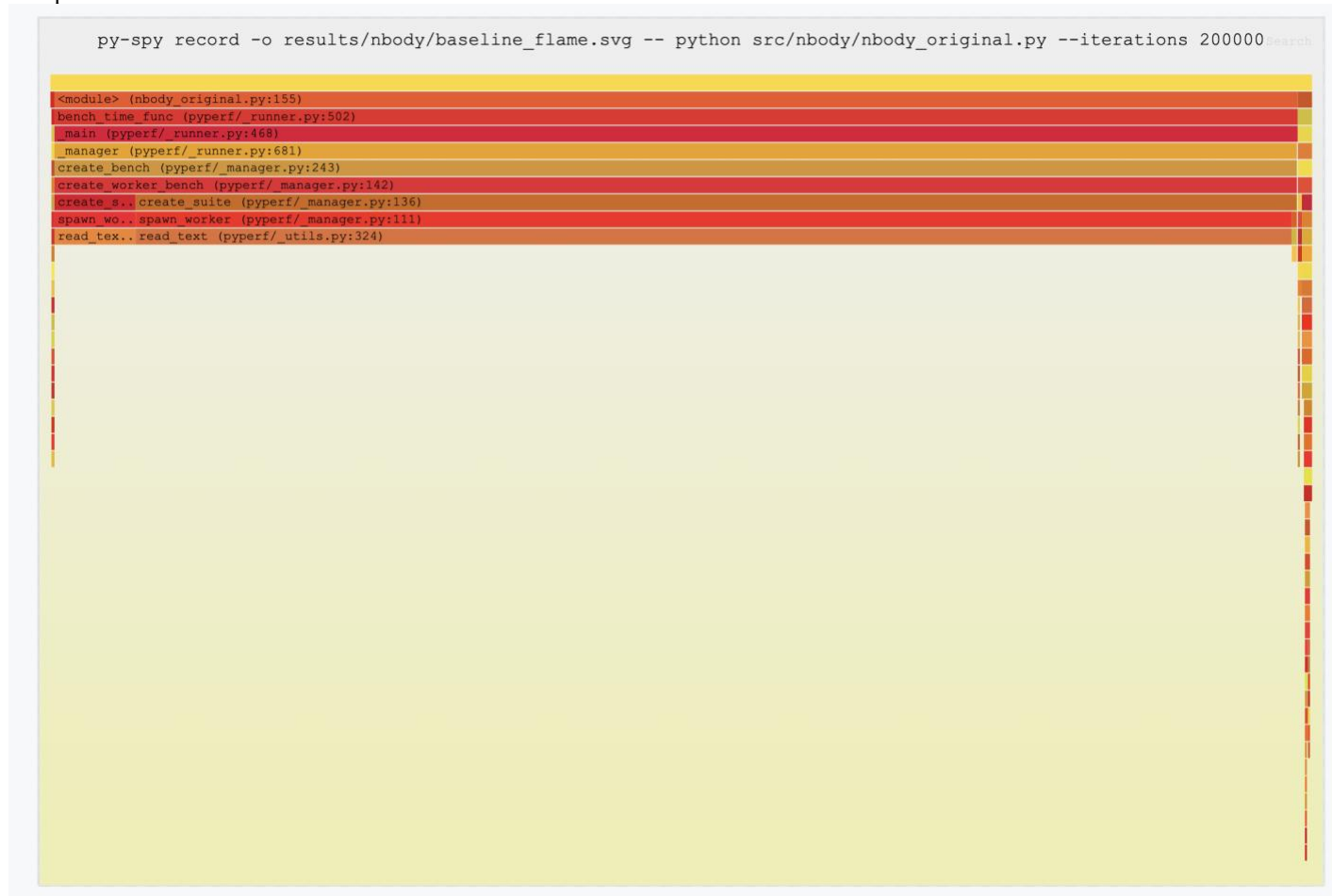
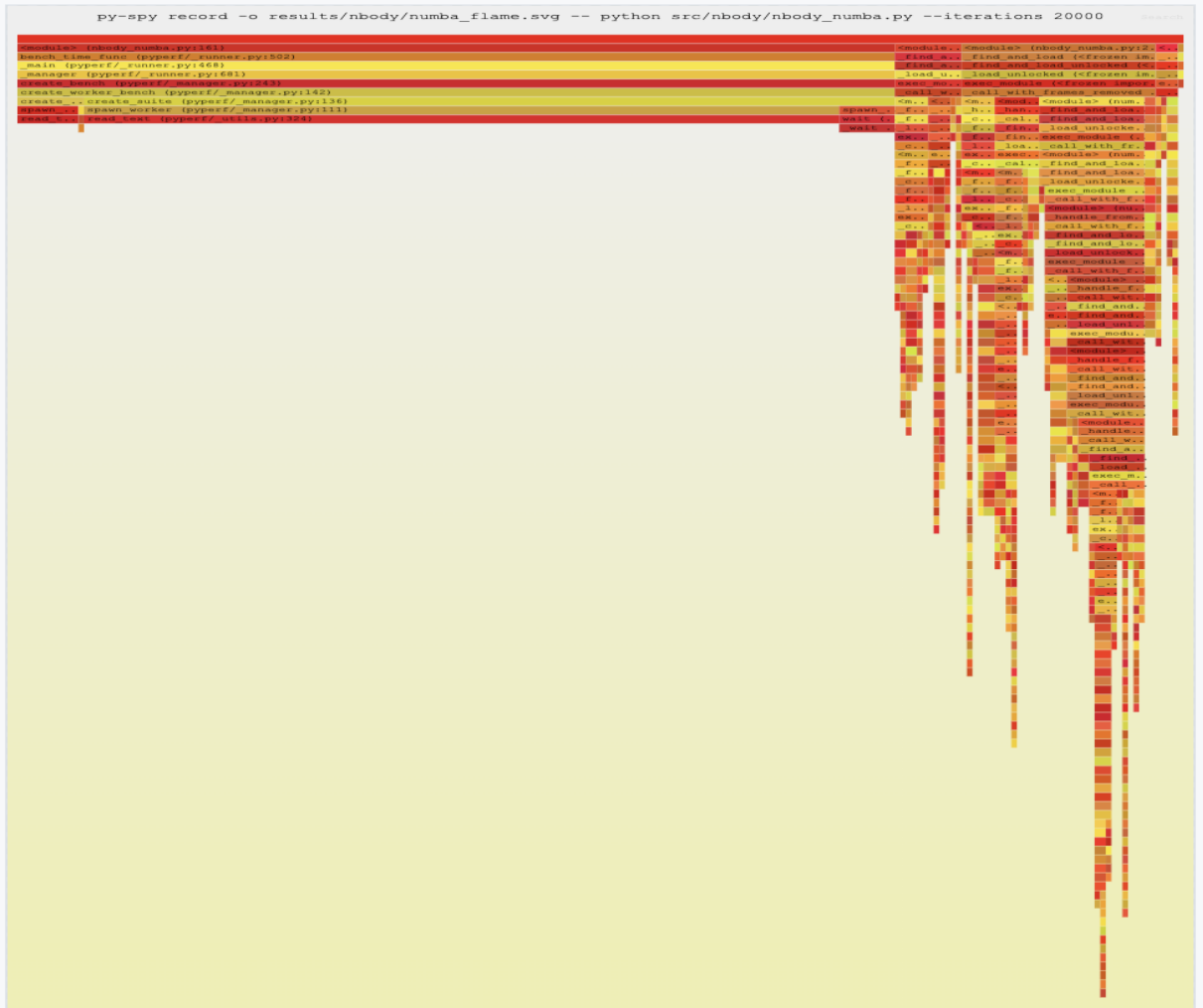


Figure 2. Flame graph of the Numba-optimized `nbody` benchmark. The main numerical work is moved out of the interpreted Python hot path.



5. Hardware Acceleration Proposal

5.1 Target

The entire `advance()` function is targeted for hardware acceleration, not just a single pairwise force computation. For each of the 20,000 simulation steps, `advance()` computes, for every one of the 10 unique body pairs:

dx, dy, dz ; $r^2 = dx^2 + dy^2 + dz^2$; $mag = dt \cdot r^{2-1.5}$; velocity updates for both bodies. Once per step, every body's position is updated from its (now-updated) velocity. This is branch-free, has a fixed small working set (5 bodies), and is $O(N^2)$ pairs per step.

5.2 Proposed architecture

Implemented in SystemVerilog as three modules (`nbody_datapath.sv`, `nbody_controller.sv`, `nbody_accelerator.sv`), plus a testbench (`nbody_tb.sv`) driving the same solar-system body values used in the software benchmark.

nbody_datapath: holds the working state (position, velocity, mass for all 5 bodies) in local registers, loaded once via `load_inputs`. On `calc_pair`, it computes one pair's full force update and signals `datapath_done`. On `update_positions`, it applies $r += dt * v$ to every body in one step and signals `datapath_done` again.

(The current datapath is a behavioral architectural model. Its floating-point operations would need to be replaced by synthesizable floating-point units in a physical implementation.)

nbody_controller is a 7-state FSM that drives two nested loops entirely in hardware: the inner loop steps through all 10 body-pair index combinations in exactly the same order as the original Python combinations()-based pairs list; the outer loop repeats this, plus a position update step, iterations times - a single hardware invocation performs the entire simulation run, not just one pair or one step.

nbody_accelerator is the top-level module wiring the above two together.

Processing is explicitly sequential, not a multi-stage overlapped pipeline: the controller asserts `calc_pair` for one pair, waits for `datapath_done`, and only then issues the next pair. This is a deliberate simplicity trade-off: consecutive pairs frequently share a body (e.g. every one of the first four pairs involves the sun), so overlapping their processing without additional hazard-detection/forwarding logic on the shared velocity registers would risk incorrect results.

5.3 Inputs / outputs

Inputs: for each of the 5 bodies, position, velocity and mass as 64-bit IEEE-754 double bit patterns; the integration timestep dt (64-bit double); and the iteration count (32-bit unsigned) - all loaded once at the start of a run. Outputs: final position and velocity for each body, as 64-bit IEEE-754 double bit patterns, valid once done is asserted. Control interface: start, busy, done. Target operating frequency is not meaningfully specifiable for the current behavioral (real-arithmetic) datapath, since the simulation-only constructs have no synthesized gate delay.

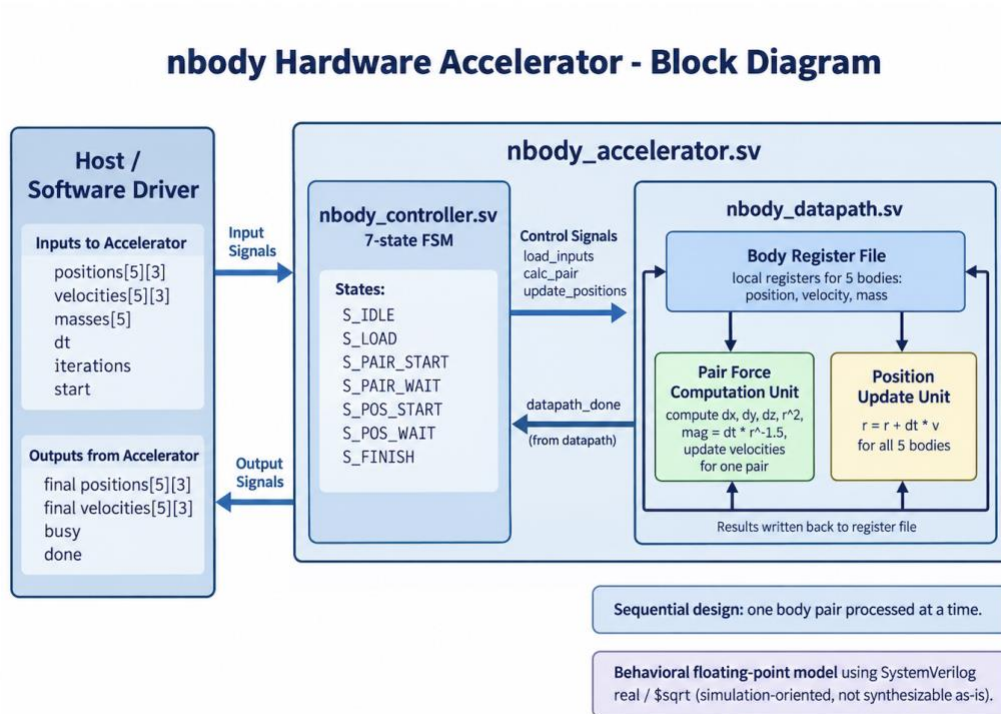
5.4 Hardware/software interface

The host writes the position/velocity/mass arrays, dt , and iterations into the accelerator's input ports, then pulses start. The controller runs the full nested loop autonomously, with no further host interaction required until completion - one hardware invocation replaces the entire 20,000-iteration `advance()` call. On completion, the host reads back the final position/velocity arrays. A minimal software driver would expose this as a single function call, e.g. `hw_advance(dt, positions, velocities, masses, iterations)`, mirroring how the numba-compiled `advance()` already replaces the pure-Python version at the software level.

5.5 Acceleration justification

`advance()` performs the same pairwise computation repeatedly for 20,000 iterations, making it a natural candidate for dedicated hardware. The accelerator keeps the five-body state locally and performs the complete simulation autonomously after it is started. The current hardware design is a proposed architecture- no hardware execution time measurement is reported.

5.6 Block diagram



5.7 Performance / area / power trade-offs

The current hardware design favors simplicity over maximum performance.

64-bit floating point.

The accelerator uses 64-bit floating-point values to match Python's numerical precision. This improves accuracy, but requires larger and more power-hungry arithmetic units than a 32-bit implementation.

Sequential processing.

The current design processes one body pair at a time. This keeps the controller and datapath simple and avoids conflicts when consecutive pairs update the same body's velocity. However, it limits throughput.

A future pipelined or parallel design could process pair calculations faster, but would require more hardware resources and additional logic to handle data dependencies.

Behavioral arithmetic.

The current SystemVerilog implementation uses behavioral floating-point operations for architectural modeling. A synthesizable implementation would require real floating-point adders, multipliers, and square-root/inverse-distance units, increasing area, latency, and power consumption.

Host communication.

The complete simulation runs inside the accelerator after `start`, so host communication is needed only before and after the run. This reduces communication overhead but gives the host less access to intermediate states.

6. Conclusion

The nbody benchmark showed that small changes to the Python code were not enough to improve performance. Manual loop unrolling produced no significant improvement, while replacing $** (-1.5)$ with *sqrt()* made the program slightly slower.

The main improvement came from Numba JIT compilation, which reduced execution time from about **229 ms to 9.24 ms**, giving approximately a **25× speedup**. This confirmed that CPython interpreter overhead was the main bottleneck.

Based on the repeated pairwise computation in *advance()*, we also designed a dedicated hardware accelerator. The current design processes body pairs sequentially and stores the five-body state locally. It prioritizes a simple and clear architecture, while pipelining or parallel pair processing could be explored as future improvements.

The SystemVerilog design is an architectural model and was not synthesized or physically implemented.